

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.sparse import lil_matrix
4 from scipy.sparse.linalg import spsolve
5 from scipy.integrate import quad

```

```

1 # gaussian quadrature
2 def gaussian_qua(func):
3     return (1/2)*(func((1/2)-(1/2)*np.sqrt(1/3))+ func((1/2)+(1/2)*np.sqrt(1/3)))
4
5 # function kapa
6 def kappa(xi,h,x_j_m_1):
7     return np.cos(2*np.pi*(h*xi+x_j_m_1))
8
9 # function f*(1-xi) , f_1
10 def f_1(xi,h,x_j_m_1):
11     return 4*np.pi*np.sin(2*np.pi*(h*xi+x_j_m_1))*np.cos(2*np.pi*(h*xi+x_j_m_1))*(1-xi)
12
13 # function f*(xi) , f_2
14 def f_2(xi,h,x_j_m_1):
15     return 4*np.pi*np.sin(2*np.pi*(h*xi+x_j_m_1))*np.cos(2*np.pi*(h*xi+x_j_m_1))*(xi)
16
17
18 def local_1(xi):
19     return 1-xi
20
21 def local_2(xi):
22     return xi
23
24 # Basis function j , j = 0, 1, ....., N, on
25 def basis_function(j, x, N):
26     # mesh
27     x_mesh = np.linspace(0, 1, N+1)
28     #equal spaced
29     h = 1 / N
30
31     if j > 0:
32         x_j_m_1 = x_mesh[j-1] # x_{j-1}
33     else:
34         x_j_m_1 = 0
35
36     x_j = x_mesh[j] # x_j
37
38     if j < N:
39         x_j_p_1 = x_mesh[j+1] # x_{j+1}
40     else:
41         x_j_p_1 = 1
42
43     if np.isscalar(x):
44         if j == 0 and x_j <= x <= x_j_p_1: # j = 0
45             return (x_j_p_1 - x) / h
46         elif j == N and x_j_m_1 <= x <= x_j: # j = N
47             return (x - x_j_m_1) / h
48         elif x_j_m_1 <= x <= x_j:
49             return (x - x_j_m_1) / h
50         elif x_j <= x <= x_j_p_1:
51             return (x_j_p_1 - x) / h
52         else:
53             return 0
54     else:
55         phi_value = np.zeros_like(x)
56
57         if j == 0: # the first basis function
58             K_j_1 = (x >= x_j) & (x <= x_j_p_1)
59             phi_value[K_j_1] = (x_j_p_1 - x[K_j_1]) / h
60         elif j == N: # the last basis function

```

```

61 .....K_j = (x >= x_j_m_1) & (x <= x_j)
62 .....phi_value[K_j] = (x[K_j] - x_j_m_1) / h
63 .....else:
64 .....K_j = (x >= x_j_m_1) & (x <= x_j)
65 .....K_j_1 = (x >= x_j) & (x <= x_j_p_1)
66 .....phi_value[K_j] = (x[K_j] - x_j_m_1) / h
67 .....phi_value[K_j_1] = (x_j_p_1 - x[K_j_1]) / h
68
69 .....return phi_value
70
71

```

```

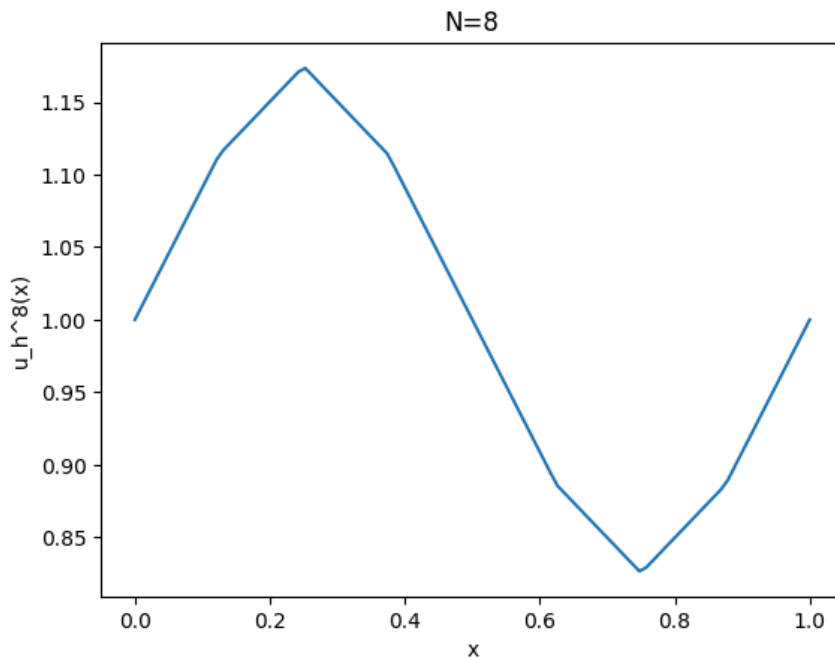
1 def FEM_solution(N,x):
2
3 # elements K_j , j = 1,...,N , x_j_m_1 = x_mesh[j-1]
4 x_mesh = np.linspace(0,1,N+1)
5
6 # equal spaced mesh
7 h = 1/N
8
9 # matrix A
10 A = lil_matrix((N, N))
11
12 #fill matrix A
13 for i in range(N):
14     if i == 0:
15         A[i,0] = N * gaussian_qua(lambda xi: kappa(xi, h=1/N, x_j_m_1= x_mesh[0]))
16         A[i,1] = -N * gaussian_qua(lambda xi: kappa(xi, h=1/N, x_j_m_1= x_mesh[0]))
17     elif i == N-1:
18         A[i,N-2] = -N * gaussian_qua(lambda xi: kappa(xi, h=1/N, x_j_m_1= x_mesh[N-1-1]))
19         A[i,N-1] = N * gaussian_qua(lambda xi: kappa(xi, h=1/N, x_j_m_1= x_mesh[N-1-1])) + \
20             N * gaussian_qua(lambda xi: kappa(xi, h=1/N, x_j_m_1= x_mesh[N-1]))
21     else:
22         A[i,i-1] = -N * gaussian_qua(lambda xi: kappa(xi, h=1/N, x_j_m_1= x_mesh[i-1]))
23         A[i,i] = N * gaussian_qua(lambda xi: kappa(xi, h=1/N, x_j_m_1= x_mesh[i-1])) + \
24             N * gaussian_qua(lambda xi: kappa(xi, h=1/N, x_j_m_1= x_mesh[i]))
25         A[i,i+1] = -N * gaussian_qua(lambda xi: kappa(xi, h=1/N, x_j_m_1= x_mesh[i]))
26 A = A.tocsr()
27 # vector F
28 F = lil_matrix((N, 1))
29
30 # Fill vector F
31 for i in range(N):
32     if i == 0:
33         F[i, 0] = h * gaussian_qua(lambda xi: f_1(xi, h=1/N, x_j_m_1=x_mesh[0])) - 1
34     elif i == N - 1:
35         F[i, 0] = h * gaussian_qua(lambda xi: f_2(xi, h=1/N, x_j_m_1=x_mesh[i - 1])) + \
36             h * gaussian_qua(lambda xi: f_1(xi, h=1/N, x_j_m_1=x_mesh[i])) + \
37             N * gaussian_qua(lambda xi: kappa(xi, h=1/N, x_j_m_1=x_mesh[i]))
38     else:
39         F[i, 0] = h * gaussian_qua(lambda xi: f_2(xi, h=1/N, x_j_m_1=x_mesh[i - 1])) + \
40             h * gaussian_qua(lambda xi: f_1(xi, h=1/N, x_j_m_1=x_mesh[i]))
41
42 # Solve AU = F
43 F.toarray()
44 U = spsolve(A, F)
45
46
47 # u_h^{N} solution
48 phi_sum = sum(U[j] * basis_function(j, x, N) for j in range(N))
49 G = basis_function(N, x, N)
50
51 return G + phi_sum , U
52
53
54

```

```

1 N = 8
2 # x in (0,1)
3 x_value = np.linspace(0, 1, 100)
4
5 plt.plot(x_value, FEM_solution(N,x_value)[0])
6
7 plt.title(f"N={N}")
8 plt.xlabel("x")
9 plt.ylabel(f"u_h^{N}(x)")
10 plt.show()

```

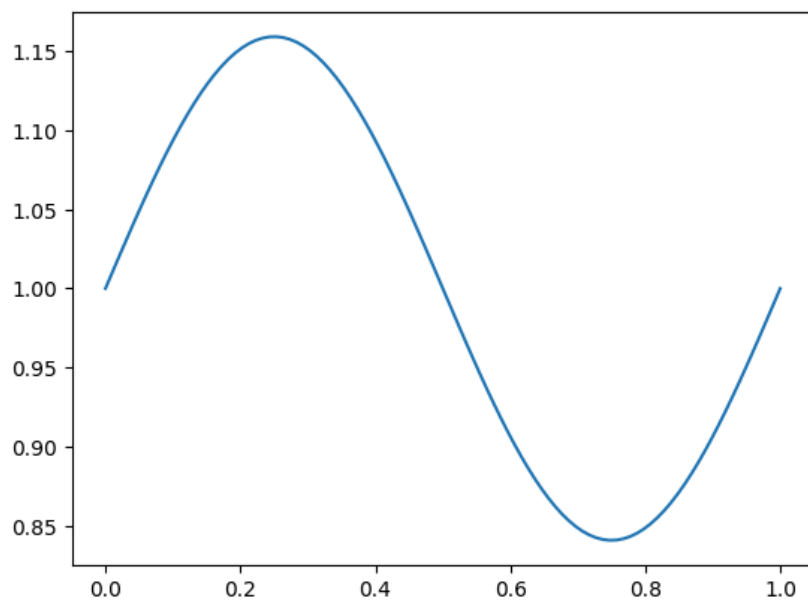


```

1 # x in (0,1)
2 x_value = np.linspace(0, 1,100)
3 exact_sol = (1 / (2 * np.pi)) * np.sin(2 * np.pi * x_value) + 1
4
5 plt.plot(x_value,exact_sol)

```

[<matplotlib.lines.Line2D at 0x78c78d05b5e0>]



```

1 N_list = [8, 16, 32, 64, 128, 256, 512]
2 error = {}

```

```

3
4 def u_exact(xi,h,x_j_m_1):
5     return (1 / (2 * np.pi)) * np.sin(2 * np.pi * (h*xi+x_j_m_1)) + 1
6
7 for item in N_list:
8     U = np.append(FEM_solution(item,x_value)[1],[1])
9
10    error_in_element = []
11    x_mesh = np.linspace(0, 1, item + 1)
12    for k in range(1,item+1):
13        error_in_element.append((1/item)*gaussian_qua(lambda xi:((U[k-1]*(1-xi)+ U[k]*xi) - u_exact
14
15    error_value = np.sqrt(sum(error_in_element))
16    error[f"N={item}"] = error_value
17

```

```

1 for item in N_list:
2     print(f"N={item} , L^2 error={error[f'N={item}']}")

```

```

N=8 , L^2 error=0.0032565958873056063
N=16 , L^2 error=0.0010929453405421877
N=32 , L^2 error=0.0003111183694536777
N=64 , L^2 error=8.258330167536194e-05
N=128 , L^2 error=2.1247405456135773e-05
N=256 , L^2 error=5.387025435866702e-06
N=512 , L^2 error=1.3561487078662786e-06

```

```

1 r_1 = (np.log(error["N=8"]/error["N=16"]))/np.log(2)
2 print(r_1)
3
4 r_2 = (np.log(error["N=16"]/error["N=32"]))/np.log(2)
5 print(r_2)
6
7 r_3 = (np.log(error["N=32"]/error["N=64"]))/np.log(2)
8 print(r_3)
9
10 r_4 = (np.log(error["N=64"]/error["N=128"]))/np.log(2)
11 print(r_4)
12
13 r_5 = (np.log(error["N=128"]/error["N=256"]))/np.log(2)
14 print(r_5)
15
16 r_6 = (np.log(error["N=256"]/error["N=512"]))/np.log(2)
17 print(r_6)
18
19

```

```

1.5751434540850748
1.8126857680013
1.9135415752165703
1.958563415586893
1.9797259006747667
1.9899734919392265

```

```

1 def FEM_solution_2(N,x):
2
3     # elements K_j , j = 1,...,N , x_j_m_1 = x_mesh[j-1]
4     x_mesh = np.linspace(0,1,N+1)
5
6     # equal spaced mesh
7     h = 1/N
8
9     # matrix A
10    A = lil_matrix((N-1, N-1))
11
12    A_11 = (1/h) - (1/2) + h/3
13    A_12 = -(1/h) + (1/2) + h/6
14    A_22 = (1/h) + (1/2) + h/3

```

```

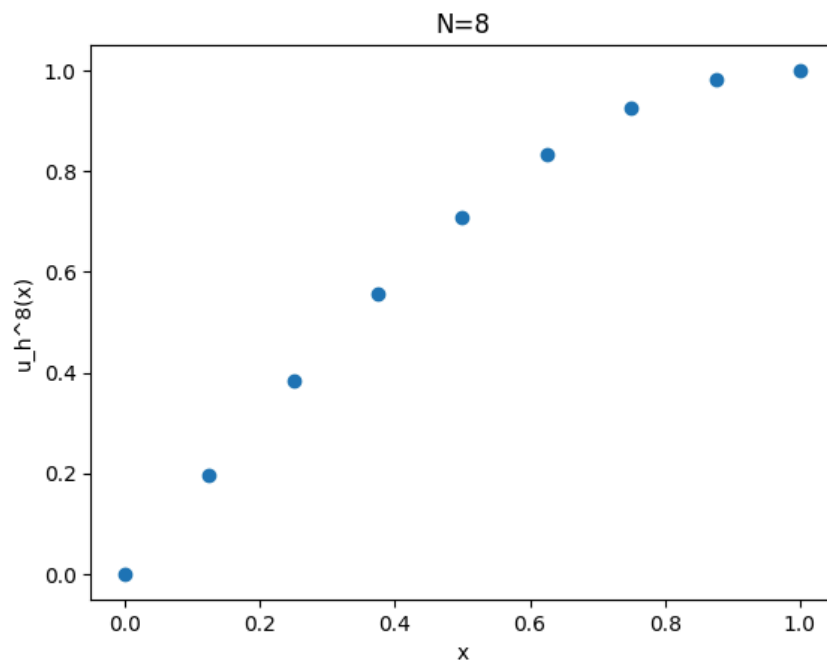
15 A_21 = -(1/h) - (1/2) + h/6
16
17 #fill matrix A
18 for i in range(N-1):
19     if i == 0:
20         A[i,0] = A_22 + A_11
21         A[i,1] = A_12
22     elif i == N-2:
23         A[i,N-3] = A_21
24         A[i,N-2] = A_22 + A_11
25     else:
26         A[i,i-1] = A_21
27         A[i,i] = A_22 + A_11
28         A[i,i+1] = A_12
29 A = A.tocsr()
30 # vector F
31 F = lil_matrix((N-1, 1))
32
33 def f(xi,h,x_j_m_1):
34     return ((1/4)*np.pi**2 + 1)*np.sin((1/2)*np.pi*(h*xi+x_j_m_1)) + (1/2)*np.pi*np.cos((1/2)*np.pi*(h*xi+x_j_m_1))
35
36 # Fill vector F
37 for i in range(N-1):
38     if i == 0:
39         F[i, 0] = h * gaussian_qua(lambda xi: f(xi, h=1/N, x_j_m_1=x_mesh[i])*local_2(xi)) + \
40             h * gaussian_qua(lambda xi: f(xi, h=1/N, x_j_m_1=x_mesh[i+1])*local_1(xi))
41     elif i == N - 2:
42         F[i, 0] = h * gaussian_qua(lambda xi: f(xi, h=1/N, x_j_m_1=x_mesh[i])*local_2(xi)) + \
43             h * gaussian_qua(lambda xi: f(xi, h=1/N, x_j_m_1=x_mesh[i+1])*local_1(xi)) - \
44             A_12
45     else:
46         F[i, 0] = h * gaussian_qua(lambda xi: f(xi, h=1/N, x_j_m_1=x_mesh[i])*local_2(xi)) + \
47             h * gaussian_qua(lambda xi: f(xi, h=1/N, x_j_m_1=x_mesh[i+1])*local_1(xi))
48
49 F.toarray()
50 #Solve AU = F
51 U = spsolve(A, F)
52
53
54 # u_h^{N} solution
55 phi_sum = sum(U[j] * basis_function(j+1, x, N) for j in range(N-1))
56 G = basis_function(N, x, N)
57
58 return G + phi_sum , U
59
60

```

```

1 N = 8
2 # x in (0,1)
3 x_value = np.linspace(0, 1, 9)
4
5 plt.plot(x_value, FEM_solution_2(N,x_value)[0],"o")
6
7 plt.title(f"N={N}")
8 plt.xlabel("x")
9 plt.ylabel(f"u_h^{N}(x)")
10 plt.show()
11 print(FEM_solution_2(N,x_value)[0])

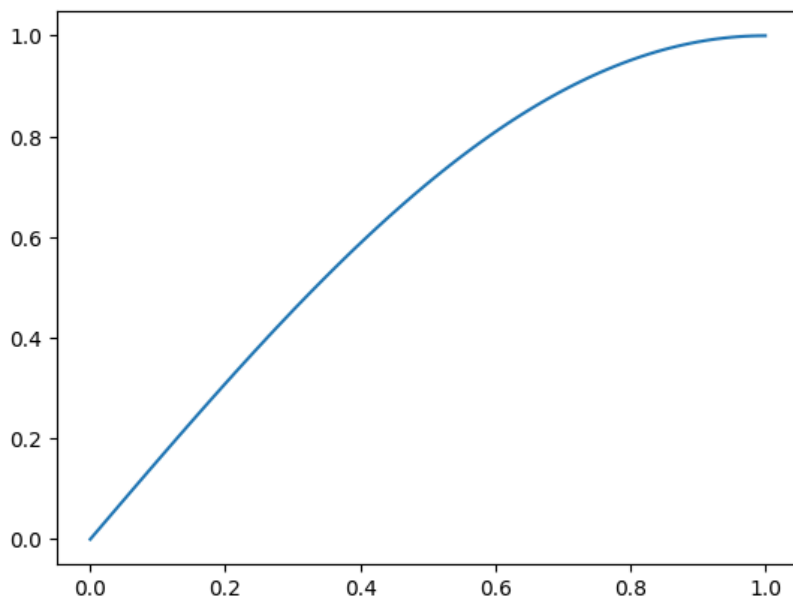
```



```
[0.          0.19533849 0.38312413 0.55613836 0.70772918 0.83206662
 0.92436651 0.98107404 1.          ]
```

```
1 # x in (0,1)
2 x_value = np.linspace(0, 1,100)
3 exact_sol_2 = np.sin((1/2)*np.pi*x_value)
4
5 plt.plot(x_value,exact_sol_2)
```

```
[<matplotlib.lines.Line2D at 0x78c78d0eb730>]
```



```
1 N_list = [8, 16, 32, 64, 128, 256, 512]
2 error = {}
3
4 def u_exact(xi, h, x_j_m_1):
5     return np.sin((1/2) * np.pi * (h * xi + x_j_m_1))
6
7 for item in N_list:
8     U = np.append(FEM_solution_2(item, x_value)[1], [1])
9     U = np.insert(U, 0, 0)
10
```

```

11 error_in_element = []
12 x_mesh = np.linspace(0, 1, item + 1)
13
14 for k in range(1, item+1):
15     error_in_element.append(
16         (1/item) * gaussian_qua(lambda xi: ((U[k-1] * (1 - xi) + U[k] * xi) -
17             u_exact(xi, 1/item, x_mesh[k-1]))**2)
18     )
19 error_value = np.sqrt(sum(error_in_element))
20 error[f"N={item}"] = error_value
21
22 print(error)
23
--

```

```
{'N=8': 0.0018951093600652494, 'N=16': 0.00047310749698965213, 'N=32': 0.00011823507287518242, 'N=64':
```

```

1 for item in N_list:
2     print(f"N={item} , L^2 error={error[f'N={item}']}")

```

```

N=8 , L^2 error=0.0018951093600652494
N=16 , L^2 error=0.00047310749698965213
N=32 , L^2 error=0.00011823507287518242
N=64 , L^2 error=2.9556156609905677e-05
N=128 , L^2 error=7.388875866434832e-06
N=256 , L^2 error=1.8472084386844682e-06
N=512 , L^2 error=4.618003646314264e-07

```

```

1 r_1 = (np.log(error["N=8"]/error["N=16"]))/np.log(2)
2 print(r_1)
3
4 r_2 = (np.log(error["N=16"]/error["N=32"]))/np.log(2)
5 print(r_2)
6
7 r_3 = (np.log(error["N=32"]/error["N=64"]))/np.log(2)
8 print(r_3)
9
10 r_4 = (np.log(error["N=64"]/error["N=128"]))/np.log(2)
11 print(r_4)
12
13 r_5 = (np.log(error["N=128"]/error["N=256"]))/np.log(2)
14 print(r_5)
15
16 r_6 = (np.log(error["N=256"]/error["N=512"]))/np.log(2)
17 print(r_6)
18

```

```

2.00204117615189
2.000509966906798
2.0001274722147615
2.0000318816236464
2.0000082224293108
2.000005451609709

```